Task 1: Реализация многопоточной версии программы умножения матрицы на вектор

M = N	Количество потоков															
	1		2		4		6		8		16		20		40	
	T	S	T	S	T	S	T	S	T	S	T	S	T	S	T	S
20000	2.41	1	1.22	1.98	0.66	3.61	0.54	4.51	0.32	7.55	0.17	14.4	0.13	17.6	0.16	14.6
40000	9.75	1	4.85	2.01	2.58	3.78	2.45	3.97	1.63	5.99	0.67	14.6	0.59	16.3	0.66	14.8

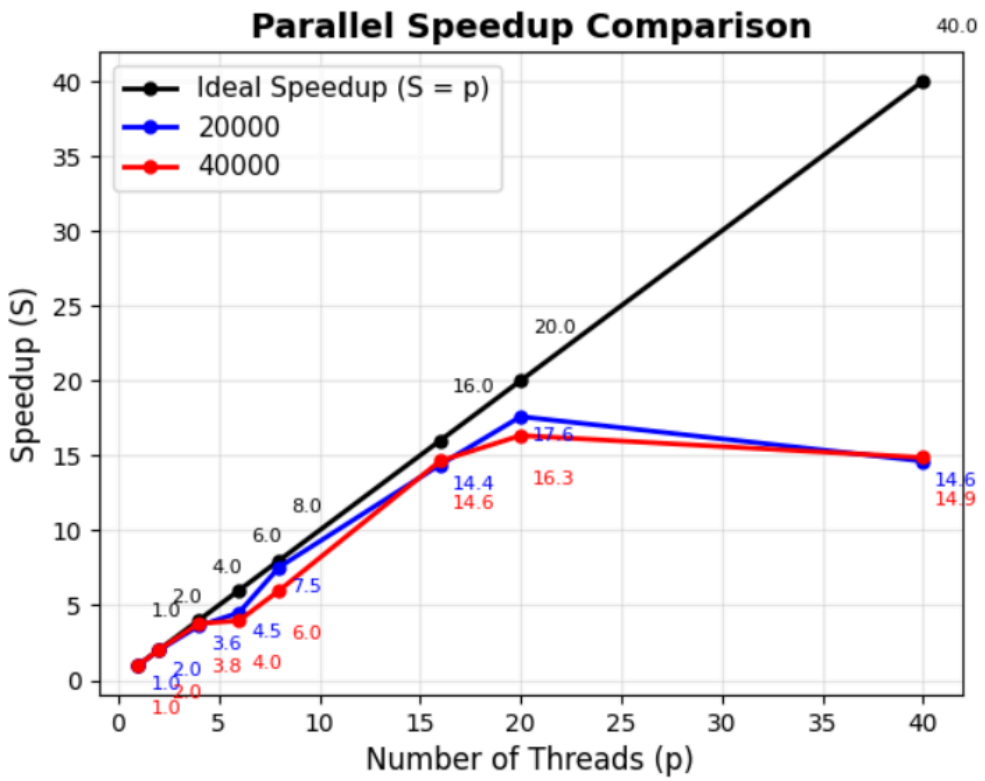
Processor:

Model name: Intel(R) Xeon(R) Gold 6248 CPU @ 2.50GHz
Architecture: x86_64
CPU(s): 80

Server name: ProLiant XL270d Gen10
NUMA nodes count: Available: 2 nodes (0-1)
Node 0 size: 385636 MB
Node 1 size: 387008 MB

Operation System:

PRETTY_NAME = "Ubuntu 22.04.5 LTS"
VERSION = "22.04.5 LTS (Jammy Jellyfish)"



Task 2: Реализация многопоточной версии программы численного интегрирования

M = N	Количество потоков															
	1		2		4		6		8		16		20		40	
	T	S	T	S	T	S	T	S	T	S	T	S	T	S	T	S
40000000	0.48	1	0.25	1.91	0.13	3.60	0.09	5.12	0.07	6.66	0.04	10.9	0.03	13.26	0.026	18.75
80000000	0.97	1	0.49	1.95	0.26	3.71	0.17	5.52	0.13	7.24	0.07	13.3	0.06	15.67	0.046	20.90

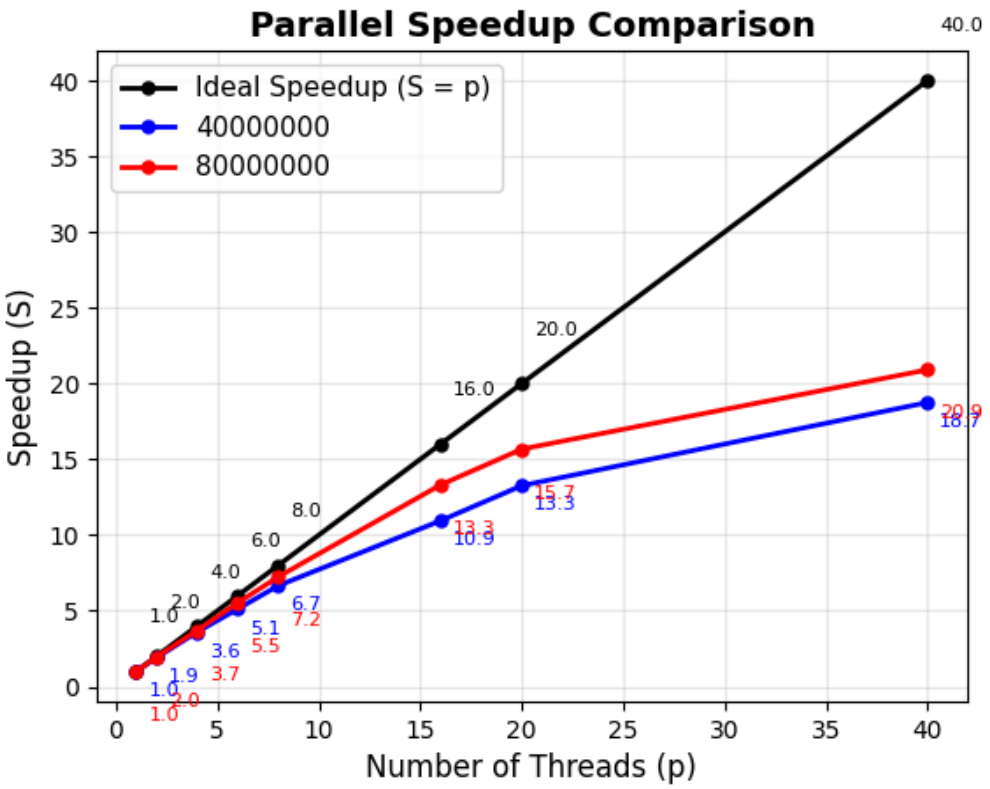
Processor:

Model name: Intel(R) Xeon(R) Gold 6248 CPU @ 2.50GHz
Architecture: x86_64
CPU(s): 80

Server name: ProLiant XL270d Gen10
NUMA nodes count: Available: 2 nodes (0-1)
Node 0 size: 385636 MB
Node 1 size: 387008 MB

Operation System:

PRETTY_NAME = "Ubuntu 22.04.5 LTS"
VERSION = "22.04.5 LTS (Jammy Jellyfish)"



Task 3: Параллельная реализация решения системы линейных алгебраических уравнений

Processor:

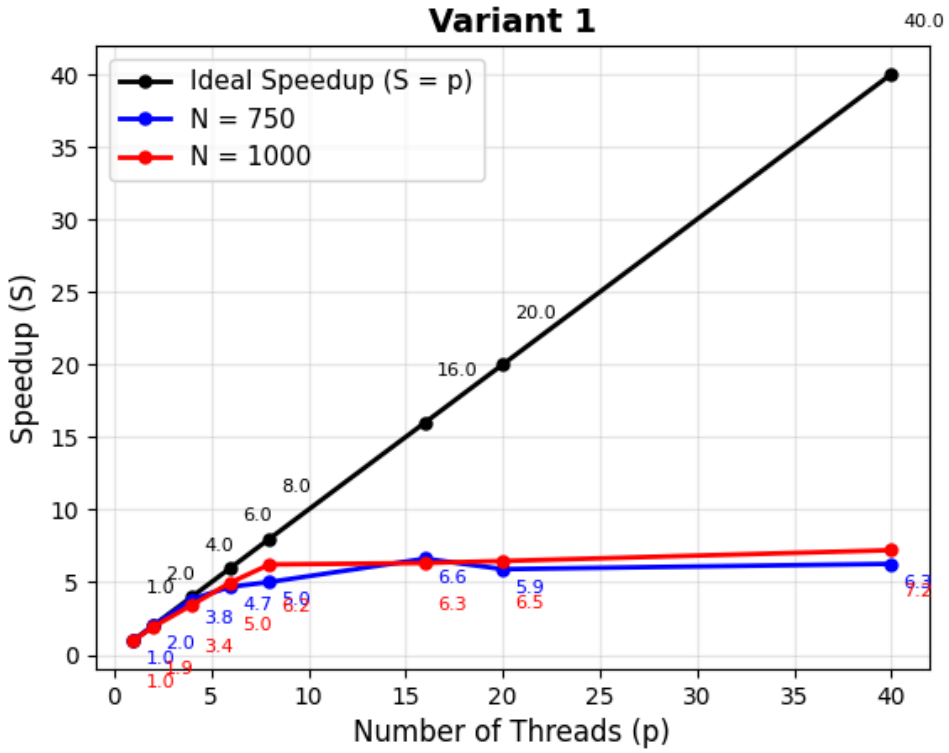
Model name: Intel(R) Xeon(R) Gold 6248 CPU @ 2.50GHz
Architecture: x86_64
CPU(s): 80

Server name: ProLiant XL270d Gen10
NUMA nodes count: Available: 2 nodes (0-1)
Node 0 size: 385636 MB
Node 1 size: 387008 MB

Operation System:
PRETTY_NAME = "Ubuntu 22.04.5 LTS"
VERSION = "22.04.5 LTS (Jammy Jellyfish)"

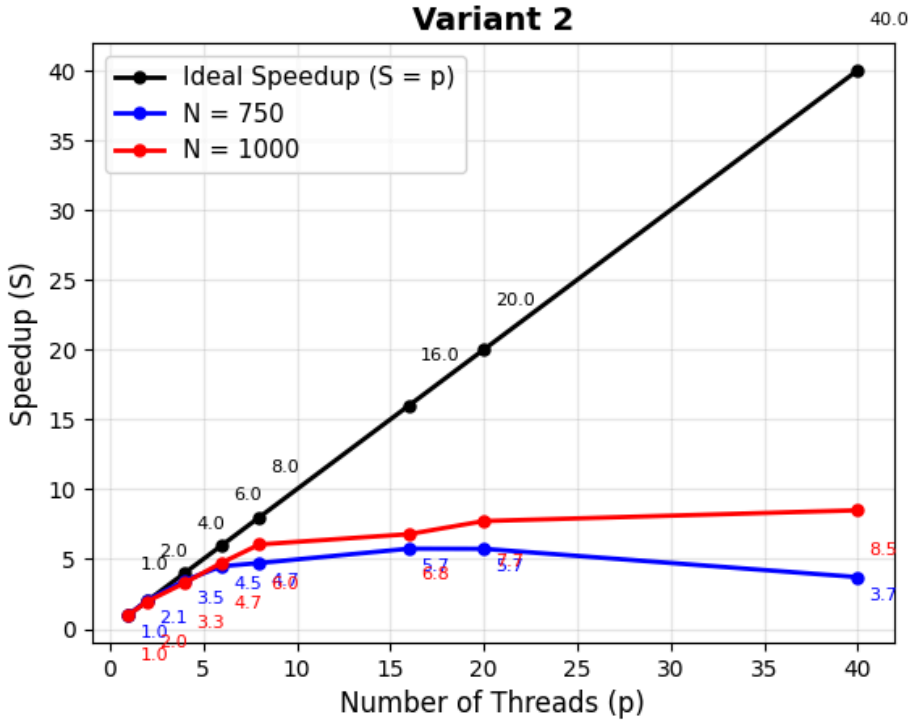
Вариант 1: Для каждого распараллеливаемого цикла создается отдельная параллельная секция #pragma omp parallel for

M = N	Количество потоков															
	1		2		4		6		8		16		20		40	
	T	S	T	S	T	S	T	S	T	S	T	S	T	S	T	S
750	35.9	1	18.1	1.99	9.39	3.83	7.68	4.69	7.19	5.01	5.44	6.62	6.10	5.89	5.74	6.27
1000	62.5	1	32.6	1.91	18.2	3.43	12.5	4.98	10.0	6.22	9.88	6.32	9.67	6.46	8.67	7.20



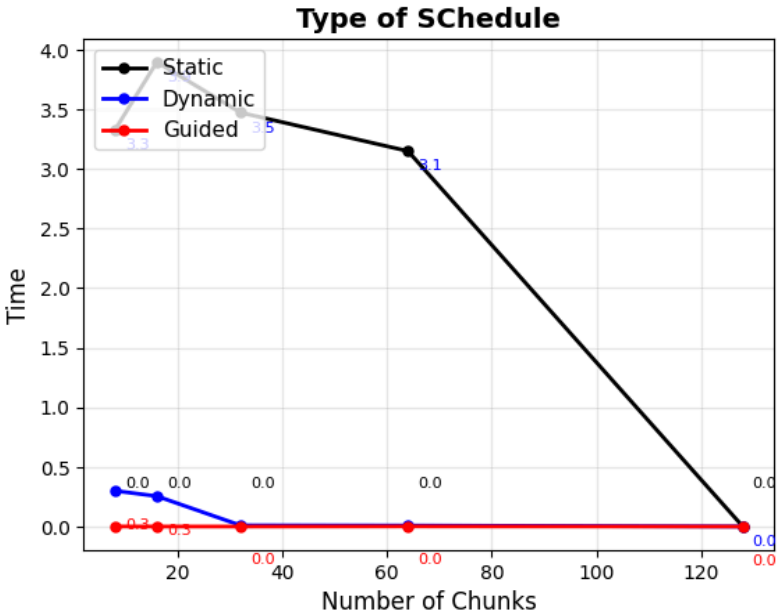
Вариант 2: Создается одна параллельная секция #pragma omp parallel, охватывающая весь итерационный алгоритм.

M = N	Количество потоков															
	1		2		4		6		8		16		20		40	
	T	S	T	S	T	S	T	S	T	S	T	S	T	S	T	S
750	35.3	1	17.2	2.05	10.2	3.46	7.87	4.49	7.49	4.72	6.15	5.74	6.17	5.73	9.53	3.71
1000	57.4	1	29.2	1.97	17.4	3.29	12.1	4.75	9.49	6.05	8.46	6.78	7.43	7.72	6.75	8.49



Вариант 3: Провести исследование на определение оптимальных параметров #pragma omp for schedule(...) при некотором фиксированном размере задачи и количестве потоков.

Type schedule	Chunks	Time(sec)
Static	8	3.326017
	16	3.894719
	32	3.471683
	64	3.148852
	128	0.000425
Dynamic	8	0.300009
	16	0.255118
	32	0.010272
	64	0.008752
	128	0.000779
Guided	8	0.001062
	16	0.000640
	32	0.000550
	64	0.001858
	128	0.000376



Вывод:

Для итерационных методов решения СЛАУ с регулярной структурой данных целесообразно использовать Вариант 2 (единая параллельная область с явной синхронизацией), так как он обеспечивает:

1. Лучшую масштабируемость: выше ускорение при большом числе потоков
2. Стабильность производительности: отсутствие резких падений при увеличении p
3. Эффективность для больших задач: при N=1000 преимущество максимально

Вариант 1 может быть рекомендован только для:
1. Прототипирования и отладки
2. Задач малого размера
3. Ситуаций, когда требуется максимальная гибкость в распределении нагрузки между этапами